AUTOMATA AND COMPILER DESIGN

III B. TECH- I SEMESTER

Course Code	Category	Hours / Week			Credits	Maximum Marks		
		L	T	P	C	CIE	SEE	Total
A5IT04	PCC	3	-	-	3	30	70	100

COURSE OBJECTIVES

1. Analyze the major concepts of language translation and phases of compiler design
2. Enumerating top down and bottom up parsing techniques used in compilation process.
3. Apply the effectiveness of optimization.
4. Introducing the syntax directed translation and type checking.
5. Illustrate the various optimization techniques for designing various optimizing compilers

COURSE OUTCOMES

1. Applying the compiler construction tools and Describes the Functionality of each stage of compilation process
2. Construct parsing tables for different types of parsing techniques and syntax directed translations.
3. Apply the different representations of intermediate code generation.
4. Apply code optimization techniques to different programming languages.
5. Generate object code for natural language representations.

UNIT – I	FORMAL LANGUAGE AND INTROUCTION TO COMPILERS	CLASSES: 10

FORMAL LANGUGAES AND REGULAR EXPRESSION: Languages, Definition: regular expressions, Finite Automata -DFA, NFA. Conversion of regular expression to NFA, Conversion of NFA to DFA, Equivalence of NFA and DFA.
INTRODUCTION TO COMPILERS: Definition of compiler, interpreter and its differences, the phases of a compiler, pass and phases of translation, bootstrapping, LEX-lexical analyzer generator.

UNIT - II	PARSERS	CLASSES: 14

PARSING: Parsing, role of parser, context free grammar, derivations, parse trees, ambiguity, elimination of left recursion, left factoring, classes of parsing, top down parsing - LL(1) grammars.
BOTTOM UP PARSING: Definition of bottom up parsing, handles, handle pruning, stack implementation of shift-reduce parsing, conflicts during shift-reduce parsing, LR grammars, LR parsers-simple LR, canonical LR (CLR) and Look Ahead LR (LALR) parsers, YACC-automatic parser generator.

UNIT – III	TYPE CHECKING	CLASSES: 12

TYPE CHECKING: Definition of type checking, type expressions, type systems, static and dynamic checking of types, specification of a simple type checker, equivalence of type expressions, type conversions.
SYNTAX DIRECTED TRANSLATION: Syntax directed definition, construction of syntax trees, S-Attributed and L-Attributed definitions.

UNIT - IV	INTERMEDIATE CODE GENERATION	CLASSES: 12

RUN TIME ENVIRONMENTS: Source language issues, Storage organization, storage-allocation Strategies, parameter passing, and symbol tables.
INTERMEDIATE CODE GENERATION: intermediate forms of source programs– abstract syntax tree, polish notation and three address code, types of three address statements and its implementation, syntax directed translation into three-address code, translation of simple statements.

UNIT - V	CODE OPTIMIZATION & GENERATION	CLASSES: 12

CODE OPTIMIZATION: basic blocks and flow graphs, optimization of basic blocks, principal sources of optimization, directed a cyclic graph (DAG) representation of basic block.
CODE GENERATION: Machine dependent code generation, object code forms, peephole optimization.

TEXT BOOKS

1. K. L. P Mishra, N. Chandrashekaran (2003), Theory of computer science- Automata Languages and computation, 2nd edition, Prentice Hall of India, New Delhi, India.
2. Alfred V. Aho, Ravi Sethi, Jeffrey D. Ullman (2007), Compilers Principles, Techniques and Tools, 2 nd edition, Pearson Education, New Delhi, India.

REFERENCE BOOKS

1. Introduction to Theory of computation.Sipser,2nd Edition, Thomson., 2009.
2. Modern Compiler Construction in C, Andrew W.Appel Cambridge University Press.
3. Kenneth C. Louden (1997), Compiler Construction– Principles and Practice, 1st edition, PWS Publishing.
4. Elements of Compiler Design, A. Meduna, Auerbach Publications, Taylor and Francis Group.
5. Principles of Compiler Design, V. Raghavan, TMH.

WEB LINKS

1. https://nptel.ac.in/courses/106/106/106106049/